Prioritized Sweeping in Maze Problems

1st Tunahan Emir Karabay
Control and Automation Engineering
Yildiz Technical University
Istanbul, Turkiye
emir.karabay@std.yildiz.edu.tr

2nd Celal Tayfur Koç
Control and Automation Engineering
Yildiz Technical University
Istanbul, Turkiye
ckoc@yildiz.edu.tr

***Abstract*—In reinforcement learning (RL), the methods used by the model to find the optimal solution are essential. An advanced planning strategy called Prioritized Sweeping (PS) is used to prioritize state transitions that has a significant value changes in order to speed up the learning process. In this work, advantages of Prioritized Sweeping over the traditional Dyna-Q method are examined by its application to a deterministic and episodic maze problem. Our simulation findings show that generally Prioritized Sweeping outperforms Dyna-Q, which updates every state-action pair with uniform random selection, in terms of learning speed and overall backup performance in the maze problem. The proposed method reduces the number of necessary updates, making it more efficient.**

***Index Terms*—Dyna-Q, prioritized sweeping, planning, maze problem, efficient learning.**

I. Introduction

Among various RL techniques, Q-learning and its variants have proven to be highly effective for solving both deterministic and episodic tasks. However, these learning algorithms may suffer from slow convergence and inefficient exploration in large state spaces. To address these issues, a class of methods that combine planning and learning has emerged, two of which are Dyna-Q and PS. Dyna-Q is a model-based Q-learning algorithm that can plan a route and reach the terminal state in an efficient way. On the other hand, PS has emerged as an enhancement to the classical Dyna-Q method by prioritizing the planning strategy based on the significance. In this work, we investigate the use of PS and Dyna-Q algorithms in a deterministic and episodic maze problem. The maze consists of an initial point, a terminal point, and various obstacles. We compare these two algorithms in terms of learning efficiency. Our aim is firsly to demonstrate the application of PS and Dyna-Q algorithms to a maze problem secondly provide that PS offers better convergence and performance compared to Dyna-Q according to number of steps until optimal solution. The environments with large state spaces and complex structures like mazes are quite convinient to test these two algorithms.

II. Literature Review

Classical methods make good use of their data but are often impractically slow, whereas techniques like prioritized sweeping hold promise for combining the advantages of efficient data utilization with faster processing. Peng and Williams (1992) developed an algorithm similar to prioritized sweeping, called Queue-Dyna, and explored a related approach derived from Dyna and Q-learning. Prioritized sweeping, in particular, focuses computing effort on the most 'interesting' aspects of the system by using rigorous accounting, thereby achieving a goal similar to that of Gauss-Seidel iteration. There are several applications of algorithms similar to prioritized sweeping have been studied; however, one of the earliest uses of this approach can be found in [1]. The generalized prioritized sweeping proposed at [2], which extends the original PS by incorporating parametric representations for both the model and value function. With model approximators like dynamic Bayesian networks, it prioritizes states based on the generalizations from the model representation. A new planning backup method called "small backup," which uses only the current value of a single successor state, making its computation time independent of the number of successor states is proposed in [4]. This approach offers more control over the planning process in model-based reinforcement learning.

III. Problem Formulation

We consider a RL problem set in a deterministic and episodic maze environment. The agent is tasked with navigating from a starting point S to a goal point G by following the shortest path. The grid world is characterized by a predetermined size and ramdomly placed obstacles, blocking the agent's movement. The grid size and obstacle positions are not altered until the episode is finilized. The movement constraints are such that the agent can only move in four directions: up, down, left, and right, and cannot pass through obstacles. Each episode begins with the agent at the initial state S, and the agent's objective is to reach the goal state G as quickly as possible. Once the target state is reached, the agent receives a reward, resets to the starting point S, and a new episode begins. The challenge lies in developing an optimal solution capable of efficiently solving different types of mazes, where the start and goal locations, grid sizes, and obstacle configurations may vary. The aim is to achieve rapid convergence to an optimal policy that can navigate the agent with minimal computational load, in different type of maze structure. Stated problem can be found in detail [3] labeled as Example 8.4. A maze sized with 6x9 states and 7 obstacles is given in Figure 1. As can be seen from the figure the agent can take action only in four direction.

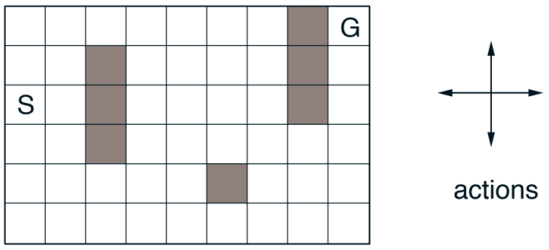


Fig. 1. Sample maze problem.

IV. Methodology

Considering the environment is determinent, the prioritized sweeping algorithm is stated in [3]. Initialize $Q(s,a)$, $Model(s,a)$, for all s,a, and $PQueue$ to empty;
Do forever:

1) $S \leftarrow$ current (nonterminal) state
2) $A \leftarrow policy(S,Q)$
3) Execute action A; observe resultant reward R, and state S'
4) $Model(S,A) \leftarrow (R,S')$
5) $P \leftarrow |R + \gamma \max_a Q(S',a) - Q(S,A)|$
6) if $P > \theta$, then insert (S,A) into $PQueue$ with priority P
7) Repeat n times, while $PQueue$ is not empty:
   a) $(S,A) \leftarrow first(PQueue)$
   b) $(R,S') \leftarrow Model(S,A)$
   c) $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
   d) Repeat, for all $\tilde{S}, \tilde{A}$ predicted to lead to S:
      i) $\tilde{R} \leftarrow$ predicted reward for $(\tilde{S}, \tilde{A}, S)$
      ii) $P \leftarrow |\tilde{R} + \gamma \max_a Q(S,a) - Q(\tilde{S}, \tilde{A})|$
      iii) if $P > \theta$ then insert $(\tilde{S}, \tilde{A})$ into $PQueue$ with priority P

Here; the S refers to the current state, S' refers to next state, A refers to the action and R refers to the recieved reward. The priority of a state-action pair is denoted by P, and $PQueue$ refers to the priority queue used to manage these priorities. Furthermore, the predicted reward, state and action is stated as $\tilde{R}$, $\tilde{S}$, $\tilde{A}$ respectively. θ is the threshold value and γ donates the discount factor.
The algorithm describes an approach for updating a Q-learning model with a planning component based on observed rewards and state transitions. Here, the Q-learning formula can be stated as:

$$Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_a Q(S',a) - Q(S,A)\right] \quad (1)$$

where α is the learning rate that controls the influence of the new information on the Q-value.
The algorithm is firstly selecting an action A according to the policy derived from the Q-values. After taking action A, observe the resulting reward R and transition to the next state S'. Store the observed reward R and the new state S' in the model for the state-action pair (S,A). Then, calculate the priority P for the state-action pair (S,A) by measuring the difference between the observed reward R and the expected Q-value update. While the priority queue is not empty, simulations are performed by retrieving the state-action pair (S,A) with the highest priority from the queue. The reward R and the resulting state S' are then retrieved from the model, and the Q-value is updated. Subsequently, for all predicted state-action pairs $(\tilde{S}, \tilde{A})$ that are expected to lead to state S, the following steps are repeated: the predicted reward $\tilde{R}$ for the transition $(\tilde{S}, \tilde{A}, S)$ is calculated, the priority for the predicted state-action pair $(\tilde{S}, \tilde{A})$ is computed, and if the priority P exceeds the threshold θ, the pair $(\tilde{S}, \tilde{A})$ is inserted into the priority queue with the calculated priority P.

V. Results

Two algorithms were utilized to solve the maze problem: DynaQ and PS. These algorithms were implemented on Python software and tested on different size of mazes with different structures. The agent's task is to reach the terminal state from a randomly positioned initial state while navigating through randomly placed obstacles. It is already expected that both of the algorithms can overcome such a deterministic problem. However, our goal is to observe how quick an agent converges to the optimal solution. The required values related to the maze environment in which the simulation was performed are presented in the form of a Table I. For both algorithms, planning number was set to 5 and each grid-world environment was simulated 50 times.

TABLE I
Maze Specifications

Grid-world Size	***Obstacles***	***Total States***	***Available States***
6x9	7	54	47
10x10	6	100	94
6x31	10	186	176
8x47	20	376	356

Moreover, the chosen coefficients and parameters are shown in Table II for a convinient simulation.

TABLE II
Simulation Parameters

γ	α	ϵ	θ	***Planning Steps***	***Episodes per grid-world***
0.95	0.1	0.1	0.1	5	10000

The beneficity can be seen from the simulation results given in Figure 2. The results demonstrate that PS converges faster than Dyna-Q. While both algorithms eventually reach the goal in all episodes, PS required fewer total Q-value updates. This is due to the prioritization of the most impactful state-action pairs in PS, which accelerates the learning process by focusing on the most critical updates. In the maze environments the agent typically receives feedback only after many steps. Dyna-Q, by updating all state-action pairs uniformly, tends to perform

inefficiently, as many updates do not contribute significantly to improving the policy. In contrast, PS effectively directs attention to high-impact state transitions, improving efficiency and reducing the number of steps to converge. Thus PS has an advatage over DynaQ algorithm in terms of elapsed time and nubmer of steps.

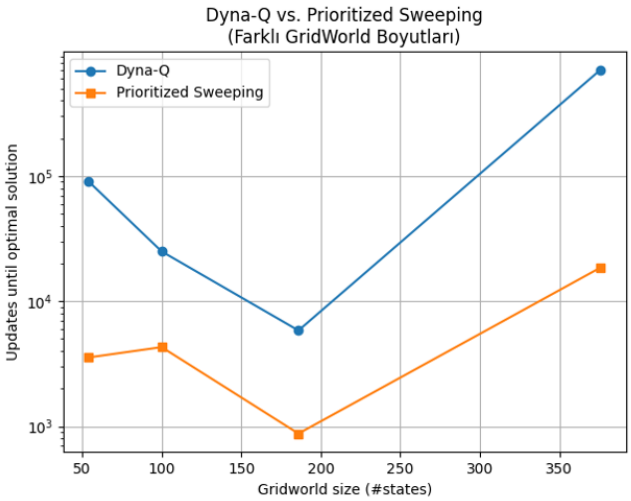


Fig. 2. Simulation results for both PS and DynaQ algorithms

VI. Conclusion

In conclusion, this study has compared the performance of PS and Dyna-Q in terms of convergence rate and steps to the optimal solution. Planning efficiency of the PS shows some advantage. Our analysis demonstrates that while both algorithms effectively update Q-values and facilitate learning in deterministic environments, they exhibit distinct differences in their convergence behaviors and computational efficiency. Normally, PS should demonstrate better performance over DynaQ for a great number of episodes, however, in our some simulations it may become reversed.

References

[1] A. W. Moore and C. G. Atkeson, 'Prioritized sweeping: Reinforcement learning with less data and less time', Machine learning, vol. 13, pp. 103–130, 1993.

[2] D. Andre, N. Friedman, and R. Parr, 'Generalized prioritized sweeping', Advances in neural information processing systems, vol. 10, 1997.

[3] R. S. Sutton, 'Reinforcement learning: An introduction', A Bradford Book, 2018.

[4] H. Van Seijen and R. Sutton, 'Planning by Prioritized Sweeping with Small Backups', in Proceedings of the 30th International Conference on Machine Learning, 17–19 Jun 2013, vol. 28, pp. 361–369.